

# Hair Cutting Robots

## Lecture 1: Get Started with Tools

Supervisor - Prof. Dr. Mohammed Aquil Mirza, COMP

Student Mentors - Mr. Ruan Haozhe, Mr. Jiang Suyu

May 2026

The Hong Kong Polytechnic University

## Announcements

1. If you are attending online, please set your ZOOM name to your full name and school name.  
e.g. "Hoshikawa, Akari (PolyU HK)" or "Yaku, Hotaru (Baptist U HK)"
2. During the lecture, please feel free to ask questions so that we can track your understanding and adjust the pace accordingly.
3. Please take notes on worksheet during the meeting itself so that you do not need to do extra works afterwards
4. You are asked to type your answers of all inclass quizzes in the chatbox of the meeting room. There will be 1-2 in class quizzes for each session.

## Projects

1. Real-Time Haptic Teleoperation System for Robotic Hair Cutting with Latency Compensation
2. Autonomous Path Planning for Robotic Hair Cutting Using Surface-Adaptive SLAM and Style Transfer Learning
3. Physics-Based Hair Simulation with Real-Time Cutting Dynamics for Robotic System Validation
4. Hybrid Control Framework for Robotic Hair Cutting: Integrating Teleoperation and Autonomous Modes
5. Safety-Certified Planning for Robotic Hair Cutting: Formal Verification of Collision Avoidance on Deformable Surfaces
6. Multi-Robot Coordination for Hair Cutting: Collaborative Strategies Using Specialized End-Effectors
7. Vision-Language-Action (VLA) Model for Autonomous Robotic Hair Cutting

# Aim

**By completing the project, we are going to**

1. To finish the project paper, and publish to a Journal.
2. For each group, fill about 2 patents with chosen topics.

## How to Choose a Topic

- Each group should choose **one** main topic from MA1–MA7.
- The listed patent directions are only examples.
- Students may propose their own patent subtopics under the chosen main topic.
- A good subtopic should define a clear technical problem, a concrete method, and a measurable advantage.

### Expected output

- One project paper direction
- About two to three patentable invention ideas per group, and apply patents out of it.

# MA1: Haptic Teleoperation

## Topic

Real-Time Haptic Teleoperation System for Robotic Hair Cutting with Latency Compensation

## Goal

Build a remote hair-cutting control system where the operator feels tool contact and the robot stays stable even when network delay changes.

## Possible patent directions

- Protected-zone micro-stroke buffering for delayed robotic hair cutting
- Delay-adaptive anatomical virtual fixtures around the ear, jawline, scalp, and neck
- Hair-contact-aware haptic event prioritization for skin contact, snagging, vibration, and protected-zone proximity

## MA2: Autonomous Path Planning

### Topic

Autonomous Path Planning for Robotic Hair Cutting Using Surface-Adaptive SLAM and Style Transfer Learning

### Goal

Create a robot that maps a person's head and hair surface, converts a target hairstyle into cutting paths, and adapts those paths to the actual head shape.

### Possible patent directions

- Visuo-tactile haircut surface mapping with RGB-D and comb or clipper contact landmarks
- Uncertainty-aware cut sequencing for uneven hair density and occluded regions
- Scalp-atlas pass-spacing adaptation using curvature, hair volume, and target blend width

## MA3: Hair Simulation

### Topic

Physics-Based Hair Simulation with Real-Time Cutting Dynamics for Robotic System Validation

### Goal

Develop a simulator where virtual hair strands react to combing, tool contact, and cutting so students can test robot control before using real hardware.

### Possible patent directions

- Haircut-specific synthetic fault injection for robot validation
- Cut-result prediction from tool angle, strand tension, and hair direction
- Hybrid-fidelity haircut simulation near the cutting tool for real-time validation



## MA4: Hybrid Control

### Topic

Hybrid Control Framework for Robotic Hair Cutting: Integrating Teleoperation and Autonomous Modes

### Goal

Design a shared-control robot where humans handle style decisions and difficult regions while the robot performs repeatable safe motions automatically.

### Possible patent directions

- Haircut-specific confidence handover between manual and autonomous control
- Operator-intent snapping for safe tapering, blending, trimming, or neckline strokes
- Demonstration-based safe stroke envelope generation from human-guided motions

## MA5: Safety-Certified Planning

### Topic

Safety-Certified Planning for Robotic Hair Cutting: Formal Verification of Collision Avoidance on Deformable Surfaces

### Goal

Build a motion planner that proves or checks that cutting tools maintain safe distances from scalp, ears, face, and neck even when the person moves slightly.

### Possible patent directions

- Fast retreat-and-resume planning after sudden client movement
- Per-pass runtime safety certificates for protected-region clearance and tool-contact limits
- Anatomy-aware deformable safety envelopes using tissue type, hair height, pose confidence, and motion uncertainty

## MA6: Multi-Robot Coordination

### Topic

Multi-Robot Coordination for Hair Cutting: Collaborative Strategies Using Specialized End-Effectors

### Goal

Coordinate multiple small robotic arms or tools so one can comb, hold, sense, or cut while avoiding conflicts near the head.

### Possible patent directions

- Comb-before-cut authorization protocol for verified hair separation and clearance
- Time-indexed scalp-zone reservation by tool hazard level and anatomical exclusion rules
- Cooperative strand-tension control between combing and cutting tools

# MA7: Vision-Language-Action Model

## Topic

Vision-Language-Action (VLA) Model for Autonomous Robotic Hair Cutting

## Goal

Build a prototype that accepts hairstyle instructions, interprets visual hair state, and generates safe high-level robot actions rather than direct uncontrolled movements.

## Possible patent directions

- Hairstyle instruction grounding into a scalp-region action plan
- Hairstyle-consistency guard for VLA-generated haircut actions
- Incremental before-and-after visual state-delta controller for small authorized passes

## Patent Subtopic Checklist

- **Specific input:** sensor data, map state, command, haptic event, or safety certificate
- **Specific method:** control rule, planning algorithm, validation procedure, or model pipeline
- **Specific output:** safer stroke, better path, clearer handover, validated action, or certified pass
- **Specific advantage:** lower risk, better quality, faster recovery, higher stability, or stronger explainability

### Reminder

Do not only describe a product idea. A patent subtopic should describe the technical mechanism that makes the product work differently.

# Patent

## **What is a patent?**

A patent is a legal right granted by a government for an invention.

## **Why is a patent useful?**

- It gives the inventor exclusive legal protection for a limited period of time
- It can create research, business, and licensing value
- It shows that the work is not just interesting, but a genuine invention

## Why Patents Are Extraordinary

Patents are **hard to get**. The standard is much higher than simply building a project that works.

In practice, a patent application also involves a lot of documentation and careful legal writing.

### A patentable invention usually must be

- **New (novel)**: it must not have been publicly disclosed before, anywhere in the world
- **Non-obvious / inventive**: it cannot be just a small or obvious change that a skilled person in that field would easily think of
- **Useful / industrially applicable**: it must do something practical and be capable of being made or used in industry

## Patentability Requirements

### There is also a legal and technical writing requirement

- **Patent-eligible subject matter:** it is generally an invention such as a process, machine, manufacture, composition of matter, or an improvement of one of these
- **Clearly describable:** the application must explain the invention well enough for others skilled in the field to understand and reproduce it
- **Clearly defined claims:** the part you want legal protection for must be stated clearly in the patent claims

So a patent is not just about building something useful. It must also be original, inventive, and written in a legally precise way.





# Introduction to Markdown

## Why Markdown?

For this project, we will use **Markdown** as the central writing format.

- Markdown is simple, readable, and easy to edit
- It helps us focus on content before final formatting
- The final submission may later be converted to  $\text{\LaTeX}$  or Word according to official regulations
- AI agents may help with formatting, conversion, and polishing in the final process

### Important

Markdown is the working format.  $\text{\LaTeX}$  or Word may be the final submission format.

# What Is Markdown?

**Markdown** is a lightweight markup language for writing structured documents.

Instead of using complex formatting menus, Markdown uses simple symbols:

- # for headings
- - for bullet lists
- **\*\*bold\*\*** for important terms
- Backticks for code, file names, and commands

The main idea is simple: write content first, then convert it into the required final format.

# Basic Essay Structure

A research essay in Markdown should have a clear structure:

```
# Project Title
Author / Group Info
## Abstract
## 1. Introduction
## 2. Related Work
## 3. Proposed Method
## 4. Evaluation Plan
## 5. Patentable Invention Ideas
## References
```

# Headings

Headings show the structure of your essay.

# Main Title

## Major Section

### Subsection

#### Smaller Subsection

## Rule

Do not choose headings only by visual size. Choose them by document hierarchy.

## Paragraphs and Lists

Paragraphs are separated by blank lines.

This is the first paragraph.

This is the second paragraph.

Use lists for requirements, design choices, steps, and summaries.

- Safety constraint
- Mapping module
- Cutting-path planner

1. Collect sensor data
2. Build the local map
3. Generate the next safe stroke

## Emphasis and Code

Markdown can mark important terms and technical text clearly.

The system uses **latency compensation**.

The robot must avoid *protected anatomy*.

The variable 'head\_pose' stores the estimated head position.

Use code blocks for algorithms, commands, configuration, or short program examples.

## Links, Images, and Tables

Markdown also supports references, diagrams, and comparisons.

```
[IEEE Xplore](https://ieeexplore.ieee.org)
```

```
![System diagram](figures/system.png)
```

```
| Method | Advantage | Limitation |  
|---|---|---|  
| Teleoperation | Human control | Network delay |  
| Autonomy | Repeatable motion | Safety verification |
```

Use these features to make technical writing easier to review.



# Markdown Essay Template

# Title

## Abstract

Briefly summarize the problem, method, and expected contribution.

## 1. Introduction

Explain the research problem and why it matters.

## 2. Proposed System

Describe the modules, inputs, outputs, and workflow.

## 3. Patentable Ideas

List two possible inventions and explain the technical mechanism.

## References

List papers, patents, datasets, or tools used.

## Good Markdown Writing Habits

- Keep one main idea per paragraph
- Use headings consistently
- Use tables only when comparison is clearer than prose
- Keep image files organized in a clear folder such as `figures/`
- Make the document readable even before conversion

### Reminder

AI agents can help with formatting and conversion, but students are responsible for the technical content.



# Introduction to GitHub

## Why GitHub?

GitHub helps a group manage a project over time.

- Store essays, code, figures, references, and notes in one place
- Track what changed, who changed it, and when
- Collaborate without sending many file versions by email
- Give supervisors and AI agents a clear project context

### **Main idea**

Markdown gives us a writing format. GitHub gives us a collaboration and history system.

## Git, GitHub, and Repository

### **Git**

A version-control tool that records project history.

### **GitHub**

An online platform for hosting Git repositories.

### **Repository**

A project folder with files, history, branches, and collaboration tools.

## Project Repository Structure

```
project-name/  
  README.md  
  essay.md  
  Cargo.toml  
  Cargo.lock  
  .gitignore  
  src/  
    main.rs  
  figures/  
  references/  
  notes/
```

### Rule

Keep writing, code, figures, and references organized from the beginning.

## README.md

README.md is the front page of the repository.

It should explain:

- Project title and group members
- Chosen MA topic
- Project goal
- Setup command
- Run command
- Current progress

## Using .gitignore

.gitignore tells Git which files should not be committed.

Generated build outputs should stay out of the repository.

```
/target/  
.DS_Store  
.vscode/  
*.swp
```

### Why?

target/ can be very large and can be rebuilt. Commit source files, not generated build folders.



## Core GitHub Workflow

```
git status
```

```
git pull
```

```
# edit files
```

```
git add README.md essay.md Cargo.toml Cargo.lock src/main.rs .gitignore
```

```
git commit -m "Add initial project structure"
```

```
git push
```

### Habit

Pull before working. Commit after a meaningful change. Push so teammates can see it.

# Commits

A commit is a saved checkpoint in the project history.

## Good commit messages

- Add initial essay outline
- Implement task queue prototype
- Add system architecture diagram
- Update patent idea section

Avoid vague messages such as `update`, `fix`, or `final final`.

# Branches and Pull Requests

## Branches

Use branches for larger changes before merging into `main`.

- `essay-draft`
- `prototype`
- `patent-ideas`

## Pull requests

Use pull requests to review changes before merging. They are useful for group review, supervisor feedback, and AI-agent review.

## Working With AI Agents

AI agents can help with:

- Revising Markdown structure
- Explaining build or code errors
- Checking whether files are organized clearly
- Converting the final document to  $\text{\LaTeX}$  or Word when required

### Important

Students must verify the technical content, claims, references, and final submission requirements.

## Good Repository Habits

- Commit often, but only after meaningful changes
- Keep filenames clear and consistent
- Keep figures in `figures/`
- Keep references in `references/`
- Do not commit `target/`
- Pull before editing shared files



# Functional Programming

## Why Study This?

So far, we have treated a function like a **black box**:

input  $\implies$  function  $\implies$  output

That is useful when we only care about what a system does.

Now we want to look **inside the black box** and ask:

- How can a function be written?
- What is the internal structure of a program?
- Why are functions so important in programming languages?

# Function

## **Extensional definition**

A function is defined by explicitly listing its input-output pairs.

This style is common in set theory and other areas of mathematics.

## **Intensional definition**

A function is defined by a rule that explains how to compute the output from the input.

This style is common in computer science.



## Example: Double

### Extensional definition

For the function that doubles a number, we can write

$$double = \{(1, 2), (2, 4), (3, 6), \dots\}$$

### Intensional definition

We can also write the rule directly:

$$double(x) = 2x$$

## AIoT Example: Smart Lamp Rule

Consider a light sensor reading called *lux*.

We can define a function that decides whether the lamp should turn on:

$$\text{turn\_on}(\text{lux}) = \begin{cases} \text{true} & \text{if } \text{lux} < 200 \\ \text{false} & \text{otherwise} \end{cases}$$

This is still a function. Input is a sensor reading. Output is an on/off decision

### Note

This definition is useful and intuitive, but it is not fully rigorous.

It is written in an informal mathematical style, not in a precise formal language with explicitly defined syntax and semantics.

## We Prefer Intensional Definitions

In programming, we usually prefer the **intensional** view.

We do not want to list all input-output pairs. We want a rule that can be applied to many inputs.

For example:

$$\textit{double}(x) = 2x$$

can be written in lambda notation as

$$\textit{double} = \lambda x.2x$$

## Lambda Calculus Basics

### Term

A term is a syntactic object in lambda calculus.

### A term can be

- a variable:  $x$
- an abstraction:  $\lambda x.E$
- an application:  $E_1 E_2$

### Remark

In this lecture we will use a slightly **sloppy** lambda notation: we will freely write numbers and arithmetic such as  $x + 1$ .

Mathematically, a more rigorous treatment would define pure lambda terms and then separately define encodings or extra primitives.

## Some Lambda Terms

### Examples

- $\lambda x. x + 1$
- $\lambda x. x \times 2$
- $(\lambda x. x + 1) 5$
- $(\lambda x. \lambda y. x \ y) (\lambda z. z) 4$

This notation lets us describe computation using functions and function application.

It is close to how programmers think, even though it is not the fully rigorous presentation of pure lambda calculus.

## Application, Not Multiplication

In lambda calculus,  $x\ y$  does **not** mean multiplication.

It means: apply the function  $x$  to the argument  $y$ .

$$(\lambda x. \lambda y. x\ y)\ (\lambda z. z)\ 4$$

In Python, the same idea can be written as:

```
(lambda x: (lambda y: x(y)))(lambda z: z)(4)
```

Here,  $\lambda z. z$  is the identity function, so the final result is 4.

## AIoT Example in Python

In AIoT, functions often process sensor data and make decisions.

```
decide_light = lambda lux: lux < 200  
to_fahrenheit = lambda c: 1.8 * c + 32
```

```
decide_light(150)    # True  
decide_light(350)    # False  
to_fahrenheit(25)    # 77.0
```

These are simple examples of turning raw sensor data into useful actions or values.

## Recursion and Y-Combinator

**Recursion** is when a function calls itself.

**Fixed-Point** in Functional Analysis is a value that does not change under a given function.

$$\forall L \in \Lambda, \exists M \in \Lambda, \text{s.t. } LM =_{\beta} M. x \notin FV(L)$$

$$\text{Witness: } (\lambda x. L(xx))(\lambda x. L(xx)) \rightarrow_{\beta} L((\lambda x. L(xx))(\lambda x. L(xx))) \implies LM =_{\beta} M$$

### Y-Combinator

$$Y = \lambda f. (\lambda x. f(xx))(\lambda x. f(xx))$$

$$\forall M, M \in \Lambda \implies YM =_{\beta} M(YM)$$

### Special Functions

$$\Delta = \lambda x. xx \quad \Omega = \Delta\Delta$$



## Why This Matters

### Why study lambda calculus?

- It gives a very small and precise model of computation
- It shows that functions can be treated as values
- Many programming language ideas are built on function definition and function application

Lambda calculus is not a practical language by itself, but it is a foundation for understanding programming languages.

## Python Demo

**Python does not use raw lambda-calculus syntax.**

It uses Python syntax:

$$\lambda x. x + 1 \implies \text{lambda } x: x + 1$$

```
>>> (lambda x: x + 1)(5)
```

```
6
```

```
>>> (lambda x: x * 2)(7)
```

```
14
```

```
>>> (lambda f: f(10))(lambda x: x + 3)
```

```
13
```